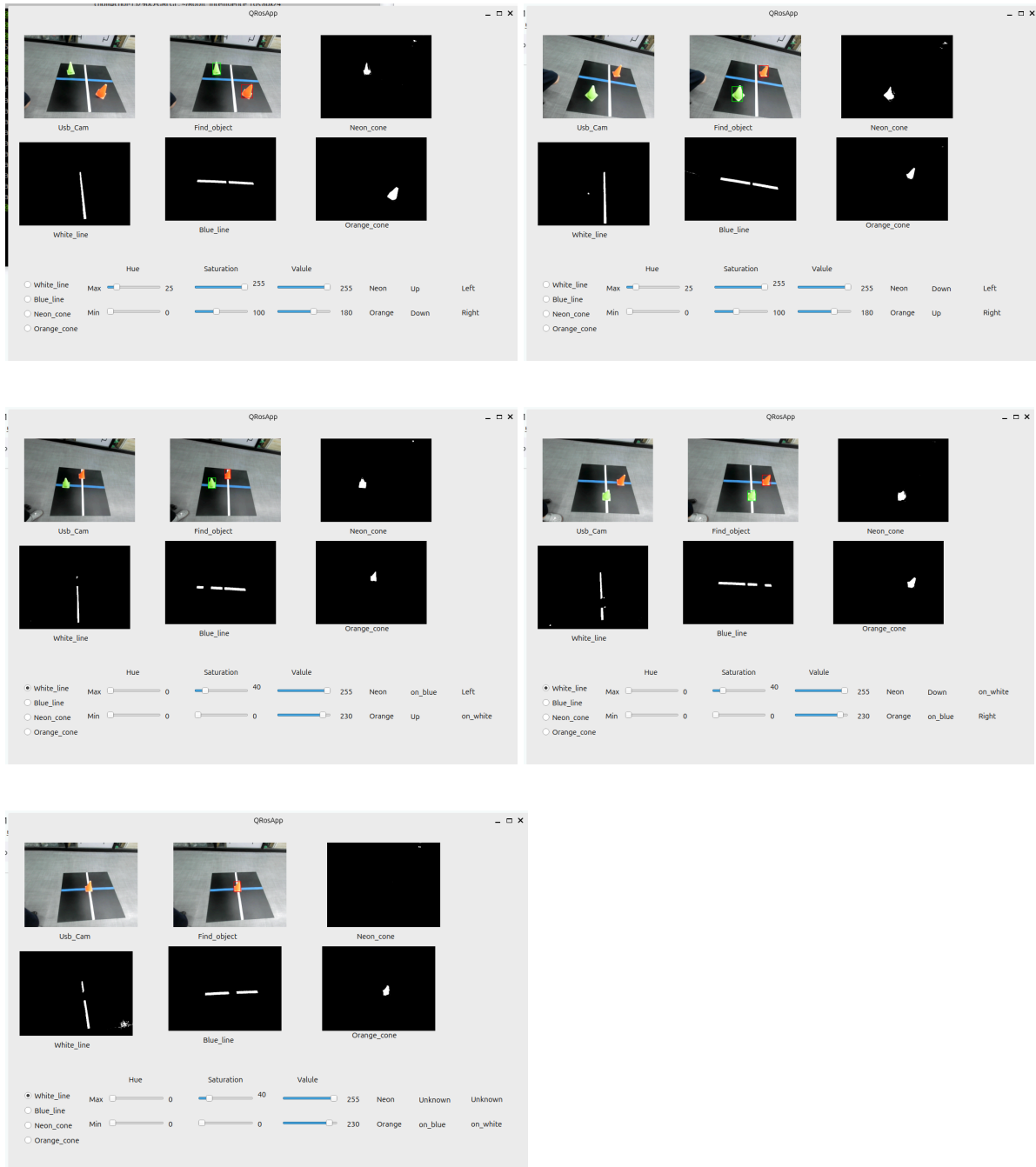


## Vision day\_1 hw2 Report

로봇 21기 예비단원 최동현



### <실행결과>

첨부할 mp4와는 다른 실행과정이지만 위 사진을 보고서에 따로 첨부하였다.

<실행결과>는 총 10가지의 위치를 표현하였다. 오른쪽 위, 왼쪽 위, 오른쪽 아래, 왼쪽 아래, 왼쪽 파란선위, 오른쪽 파란선위, 위쪽 하얀선위, 아래쪽 하얀선위, 파란선위 하얀선위, unknown이렇게 10가지를 표현하였다.

### <실행과정>

qt를 이용하여 작업하여야 하기 때문에 [https://github.com/mjlee111/ros2\\_create\\_qt\\_pkg.git](https://github.com/mjlee111/ros2_create_qt_pkg.git) 해당링크에 있는 템플릿을 깃클론하여 사용하였다.

처음에는 **label**을 이용하여 카메라가 출력되는 지 확인하였다. 데이터 프레임을 매순간 전송하여 화면에 카메라를 띄우는데 성공하였다.

하지만 다른 카메라를 사용하거나 기존카메라를 다시 끼우면 카메라를 인식하지 못하여서

```
cap.open("/dev/v4l/by-id/usb-Sonix_Technology_Co._Ltd._USB_Live_camera
_SN0001-video-index0",
         cv::CAP_V4L2);
```

이렇게 카메라 자체를 명시하여 사용하였다.

이후 **white\_line**, **blue\_line**, **Neon\_cone**, **orange\_cone**과 같이 **binary**를 해줘야하는 화면들을 **update\_frame**함수에서 처리를 해서 송출하였다.

처음에는 임의의 값을 **binary**하여서 검은 화면이 떠서 슬라이더와 라디오 버튼을 추가로 만들었다.

라디오 버튼을 통해서 어떤 화면을 조절할건지를 표시하고, 슬라이더로 **hsv**의 **max**, **min**값을 조절하여 **binary**하게 제작하였다.

이후로는 **find\_object**에서 **neon\_cone**, **orange\_cone**을 기준으로 각 콘이 어느 위치에 있는지 바운딩 박스로 표현하였다. **binary**값을 아무리 세밀하게 잡아도 미세한 노이즈들이 생길 수 있어서 각 **cone**화면기준으로 가장 큰 덩어리를 **cone**으로 간주하였다. 그 **cone**의 **width**, **height**값을 구해서 바운딩 박스의 **x,y**값으로 하여서 **find\_object**화면에 덮어씌워서 송출하였다.

마지막으로는 각 **cone**의 위치를 파란선과 하얀선을 기준으로 판단하는 **ui** 를 추가하였다. 선을 표시해주는 **white\_line**, **blue\_line** **binary**카메라를 기준으로 각 화면에서 나오는 가장 큰 덩어리를 해당 **white**, **blue** 선이라고 측정하였다. 이후 **white\_line**은 대표**x**값, **blue\_line**은 대표**y**값을 추출하였다. 왜냐하면 **white\_line**보다 왼쪽에있으면 **left**, 오른쪽에 있으면 **right**라고 판단하고, **blue\_line**보다 아래에 있으면 **down**, 위에 있으면 **up** 이라고 판단하기 위해서였다.

이런 로직으로 코드를 추가하니 생각보다 정확하게 위치가 나왔다. 다음으로는 이제 선위에 있는 것을 판단하였다. 선위에 있다고 판단하는 로직은 간단하다. **blue\_y**값이 선의 중심이라고 생각했을때 그 값을 기준으로 **-100, 100**구간은 선위라고 판단하였다. 그런데 **100**이 생각보다 너무 넓어서 나중에는 **30**으로 구간을 줄이기도 하였다. 또한 카메라의 각도가 바로 위에서 비추는 것이 아니기 때문에 **blue\_line**위에 있는 것은 추가적으로 구간을 **-30~0**으로 하였다.

```
// 위/아래 판정
if (blue_y < 0)                ud = "?";                // 기준선 못 찾음
else if (c.y >= blue_y-30 && c.y <=blue_y)            ud="on_blue";
else if (c.y < blue_y-30)      ud = "Up";
else                          ud = "Down";
// 좌/우 판정 ← 직접 채워보기 (white_x와 c.x 비교)
if (white_x < 0)               rl = "?";
else if (c.x >= white_x-30 && c.x <=white_x+30)        rl="on_white";
```

```
else if (c.x < white_x-30) rl = "Left";  
else
```

그래서 추가적인 제언으로는 **white, blue**의 대표 **x,y**값만을 추출하지 말고 길이 값까지 추출하여 카메라에서 보는 포맥스의 크기를 직사각형이나 사다리꼴로 계산한다. 이후 수식을 통해 선위에 있음을 카메라 각도가 달라짐에 따라 스스로 값을 조정해서 판단하는 로직을 구현해 보고 싶다.

## =====

### Opencv 개념 공부

#### 1. 디지털 이미지의 표현

디지털 이미지는 색을 가진 최소 단위인 픽셀들이 격자 형태로 배열된 것이다. **opencv**는 이미지를 **cv::Mat** 자료형으로 다루며, 이는 높이(행)x너비(열)x채널(channel)로 구성된 다차원 행렬이다.

채널: 컬러 이미지는 보통 3개의 채널로 구성된다. **opencv**는 관례적으로 **B-G-R**순서로 채널을 저장한다.

픽셀값: 각채널은 **0~255**의 정수 값을 가지며, 세 채널의 조합으로 하나의 색이 표현된다. 즉 **640x480** 컬러 이미지는 **640x480x3** 크기의 행렬이며, 영상 처리란 결국 이 행렬의 원소들을 규칙에 따라 계산/변형하는 과정이라 할 수 있다.

#### 2. 색공간 - RGB/HSV

특정 색을 검출할 때 **RGB** 색공간은 조명 변화에 매우 취약하다는 문제가 있다. 예를 들어 같은 빨간 물체라도 밝은곳과 어두운 곳에서 **RGB** 값이 모두 크게 달라져, 하나의 범위로 색을 지정하기 어렵다. 이 때문에 색상 검출에는 **HSV** 색공간을 주로 사용한다.

아직은 어떤것이 어느 부분에 적용될 지 잘모르겠으나, 오늘 과제에서도 측정 색을 인식하여 송출하는 **binary**를 할때 만약 **RGB**였다면 빛때문에 **RGB**개의 값의 **max, min**을 조절해 줘야 하거나, 애초에 값자체가 너무 요동쳐서 이러한 색 검출이 쉽지 않았을 수도 있다. 하지만 **hsv**에서는 **saturation**과 **value**는 어느정도 **100~250**정도로 열어둔채, 주황색을 검출하고 싶으면 **hue**값을 **10~30**으로 조절해서 **mask**를 하면 **RGB**보다는 쉽게 되는 것을 알게 되었다.

#### 3. 모폴로지 연산-노이즈 제거

**mask**에는 조명반사나 색이 비슷한 배경 때문에 작은 노이즈나 물체 내부의 구멍이 생기기 쉽다. 이를 정리하기 위해 모폴로지 연산을 사용한다.

침식: '커널이 덮은 픽셀이 전부 흰색일때만 중심을 흰색으로 남긴다.' 하나라도 검으면 중심을 깎아버린다. 그래서 흰 덩어리의 가장자리가 안쪽으로 깎이고, 커널보다 작은 흰 점은 주변이 다 흰색일 수 없으니 통째로 사라진다.

팽창: 침식의 반대이다. '커널이 덮은 픽셀 중 하나라도 흰색이면 중심을 흰색으로 만든다.' 흰색영역이 바깥으로 번지고, 흰 물체 내부의 작은 검은 구멍이나 끊긴 틈이 흰색으로 메워진다.

열림(침식->팽창): 침식으로 작은 흰점을 완전히 지워버린다. 이미 사라진 점은 이어지는 팽창으로도 살아나지 못한다. 반면 살아남은 큰 물체는 침으로 줄었다가 팽창으로 원래 크기로 회복된다. 그래서 배경의 작은 흰점을 골라 제거하고 본체는 유지시킨다.

닫힘(팽창->침식): 팽창으로 물체 내부의 작은 검은 구멍/틈을 메운다. 메워진 구멍은 이어지는 침식으로 다시 뚫리지 않는다. 바깥 경계는 팽창으로 커졌다가 침식으로 원래대로 돌아온다. 그래서 내부 구멍/끊김만 채우고 크기는 유지한다.

과제2에서 이 모폴로지 연산을 추가로 사용해본다면 **white\_line**, **blue\_line**의 선인식에 열림을 사용해 보았을 것 같다. 선을 인식하다 보면 특히 흰색선은 다른 배경들과 같이 인식되거나 아니면 빛반사로 인해 바탕의 밝은 부분이 노이즈처럼 인식되어 **cone**들의 위치 판별에 어려움을 겪기도 하였다. 이때 열림을 사용하였다면 흰색선이라고 인식되는 노이즈들을 줄이고 흰색선을 좀 더 정확하게 인식할 수 있었을 것 같다.

#### 4. ROI와 이미지 좌표계

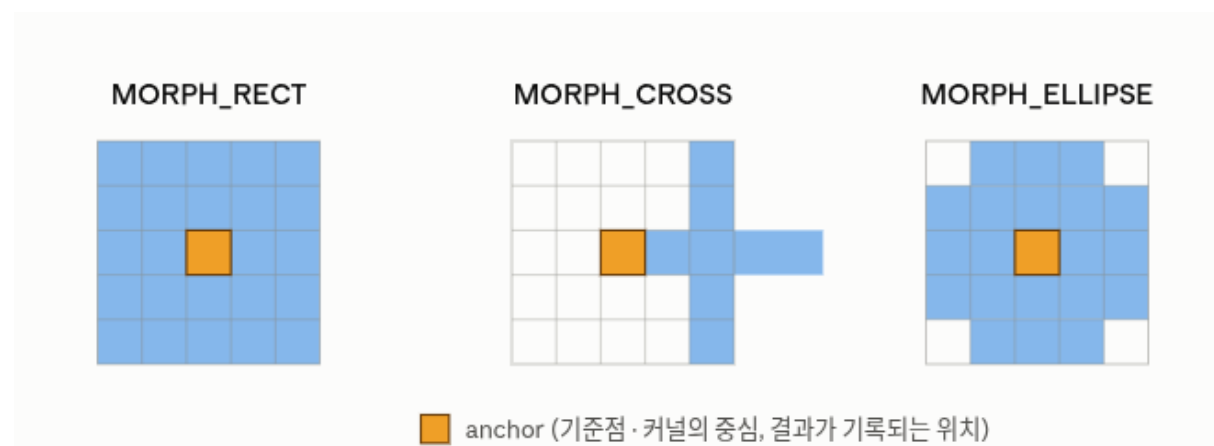
opencv의 이미지 좌표계는 왼쪽 위 모서리가 원점(0,0)이며, x는 오른쪽으로, y는 아래쪽으로 증가한다. 이는 수학의 좌표계에서 y축 방향이 반대이므로, 사분면을 판단할 때 y부호를 뒤집어 해석해야 한다는 점에 주의해야 한다. 또한 필요한 부분만 처리하기 위해 이미지의 일부 영역만 잘라 사용하는 관심영역기법을 활용하면 연산량을 줄이고 오검출을 억제할 수 있다.

위에 선을 인식하고 선의 특정 x,y값 안에 **cone**의 중심이 위치하고 있다면 선위에 있다고 판단하는 로직이 있었다. 그 코드에서도 x,y의 좌표값이 반대로 연산되고 있었다. 처음에는 반대여서 로직을 짤때 오류가 있었다. **opencv**에서는 이미지 좌표값을 반대로 사용하고 있다는 것을 새롭게 알게 되어서 보고서 내용에 추가하였다.

#### 5. 커널

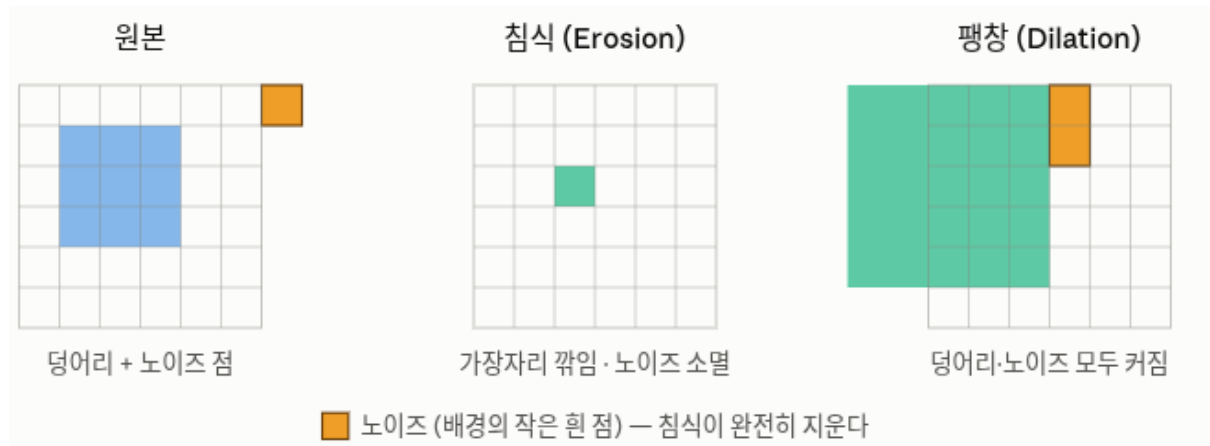
커널(kernel), 정식 명칭으로 구조 요소 (structuring element)는 이미지 위를 훑고 지나가는 작은 판이다.. 이 판에는 기준점 (anchor)이 하나 있는데, 보통 정중앙이다. 커널을 이미지의 모든 픽셀 위에 차례로 올려놓고, 그 자리에서 “이픽셀을 흰색으로 둘까 검은색으로 둘까”를 커널이 덮은 이웃들을 보고 결정한다.

커널의 핵심은 이 판이 어떤 모양인지, 그리고 기준점이 어디인지이다. **opencv**에서는 **cv::getstructuringElement**(모양, 크기)로 만들고, 모양은 세 가지가 있다. 아래는 5x5 커널의 모양별로 그린것이다. 색칠된 칸이 “이 위치의 이웃을 본다”는 뜻이고, 가운데 주황 칸이 결과가 기록되는 기준점이다.



**morph\_rect**은 이웃 전부를, **morph\_cross**는 상하좌우 십자만, **morph\_ellipse**는 원에 가까운 이웃을 본다. 콘처럼 둥근 물체엔, 타원, 각진 물체엔 사각형이 잘 맞는다.

이제 이 커널이 어떻게 판정하는지가 핵심이다. 커널을 이미지의 모든 픽셀 위에 차례로 올려놓ㄱ, 기준점이 놓인 자리를 이렇게 정한다. 침식: 커널이 덮은 칸이 전부 흰색일 때만 기준점을 흰색으로 남긴다. (**and** 규칙), 팽창: 커널이 덮은 칸 중 하나라도 흰색이면 기준점을 흰색으로 만든다. (**or** 규칙)



이 규칙을 작은 이진 이미지에 **3x3**사각형 커널로 적용하면, 어떻게 되는지 잘 보여주는 예시이다.

이 그림은 열림의 원리를 그대로 보여준다. 침식을 먼저 하면, 노이즈 점은 완전히 사라지고(주변에 흰색이 하나도 없어서), 살아남은 덩어리는 가운데 한칸으로 줄어든다. 여기서 팽창을 이어 하면, 이미 사라진 노이즈는 되살아나지 못하고, 줄어든 덩어리만 원래 크기로 회복된다. 그래서 열림 = 침식->팽창이 “본체는 지키고 배경의 작은 점만 골라 없애는” 연산인 것이다. 닫힘은 반대로 팽창을 먼저 해서 물체의 내부의 검은 구멍을 메운 뒤 침으로 크기를 되돌린다.

추가적으로 커널의 크기는 강도이다. **3x3**은 살짝, **7x7**는 세게 깎거나 넓혀진ㄱ. 노이즈보다 크고 물체보다 작은 크기를 잘 골라야 한다.